ALGORITHMIC DUNGEON

ESCAPE THE LABYRINTH

DETAILED FLUTTER IMPLEMENTATION REPORT

A detailed product, software architecture, algorithm design, implementation, testing, and delivery report

DAA FA Project | PCCOE Pune | Information Technology | Semester V

Technology: Flutter + Dart | Core algorithm: Dijkstra's shortest-path algorithm

Design direction: clean, light background; pale stone surfaces; readable dark text; restrained blue, teal, and amber accents.

Prepared as an implementation blueprint for a student development team. This document defines a practical first release, explicit rules, acceptance criteria, and a staged delivery plan.

LIGHT UI DIRECTION • MODULAR ARCHITECTURE • TESTABLE ALGORITHM

How to Use This Report

Purpose

Use this report as the team's single working specification. It is deliberately more detailed than a concept note: it describes what to build, how modules should interact, how the algorithm must behave, and how the team can prove correctness during the FA demonstration.

Recommended reading order

1. Read the product requirements and game rules before coding.
2. Implement and test the algorithm package independently.
3. Build the playable game loop and state controller.
4. Add visualization only after pathfinding tests pass.
5. Finish with usability, performance, and demonstration preparation.

Source-of-truth rule

When implementation choices conflict, prioritize the explicit game rules, then the algorithm contract, then visual preferences. Keep the algorithm independent of widgets so that the team can test it without launching Flutter.

Scope discipline: a stable, complete single-player game with one excellent level and an accurate algorithm visualization is more valuable than many unfinished levels.

Executive Summary

Algorithmic Dungeon: Escape the Labyrinth is a 2D grid-based puzzle game in which a player navigates a weighted dungeon, manages energy, collects optional treasure, and attempts to reach an exit. Every traversable tile has a non-negative entry cost. The player can explore manually or activate Algorithm Vision to watch Dijkstra's algorithm compute a minimum-cost route.

Flutter provides the cross-platform UI and rendering layer; Dart contains the dungeon model, graph construction, pathfinding engine, game rules, and statistics. CustomPainter renders the board, while Provider/ChangeNotifier or an equivalent simple state holder keeps UI updates predictable.

The educational value comes from the connection between game mechanics and algorithmic decisions. A route with fewer moves can cost more energy than a longer route. The algorithm must therefore optimize accumulated cost, not merely the number of cells.

Primary outcomes

- A playable Flutter application with responsive controls.
- A tested Dijkstra implementation using a min-priority queue.
- An animated exploration view and final path overlay.
- A results panel comparing the player's actual route cost with the optimal route cost.
- Unit tests, a reproducible demo level, and an FA presentation script.

Problem Statement and Motivation

Problem

Students often learn shortest-path algorithms through static graphs and tables. Such examples show the result but may not make the effect of weights, relaxation, visited nodes, or path reconstruction intuitive.

Proposed response

The game turns the graph into a place the player can explore. Walls represent absent vertices or edges; terrain types represent edge weights; the current player location is the source; and the exit is the target. Algorithm Vision makes the computation visible.

Why this is a DAA project

- The algorithm is central to the experience rather than decorative.
- The graph model, edge weights, relaxation rule, and complexity can be explained precisely.
- Correctness can be checked against known test maps and path costs.
- Performance can be measured over increasingly large grids.

Non-goals

The first release is not a 3D action game, online multiplayer platform, or production-grade game economy. Do not spend early time on accounts, cloud services, or advanced physics.

Product Goals, Users, and Success Criteria

Target users

The primary audience is the student team, faculty evaluator, and classmates attending the FA demonstration. The interface should be understandable without prior knowledge of graph theory, while the Algorithm Lab should provide sufficient detail for a technical viva.

Product goals

- Make a complete level playable from start to finish.
- Make movement and energy updates feel immediate and predictable.
- Show the difference between minimum steps and minimum weighted cost.
- Explain how Dijkstra reaches the answer.
- Support repeatable tests and a dependable live demo.

Acceptance criteria

- A valid movement never enters a wall or leaves the grid.
- Every valid move charges the configured entry cost exactly once.
- Dijkstra returns a minimum-cost route for all reachable test maps with non-negative weights.
- Unreachable goals are reported safely, without crashing.
- The visualized path starts at the current position and ends at the exit.
- The displayed optimal cost equals the sum of edge costs along that path.
- Restart returns the level to its initial state.

Game Concept and Narrative

Premise

The explorer is trapped in an ancient labyrinth whose navigation core can analyze the corridors. The player must reach the exit portal before their energy reserve is exhausted. Some corridors are easy to cross; rubble, water, and hazardous terrain consume more energy.

Player fantasy

The player is a clever explorer who can solve the maze manually and optionally use an ancient navigation system. Algorithm Vision should feel like a meaningful ability, not a mandatory pop-up.

Tone and presentation

Use a light, polished puzzle-game style: pale stone backgrounds, clear tile shapes, small expressive icons, and restrained accent colors. Keep all essential status information legible on a phone.

Narrative boundaries

Story text should support gameplay without delaying it. Use a short introduction, concise tooltips, and brief level-completion messages. The core demo must remain playable even if audio is disabled.

Core Gameplay Loop

1. Select a level and review its objective.
2. Inspect the visible dungeon and available energy.
3. Move one cell at a time using directional buttons or swipe controls.
4. Pay the destination tile's movement cost.
5. Collect optional treasure or energy pickups.
6. Activate Algorithm Vision to inspect a computed minimum-cost route.
7. Reach the exit or fail when the level's failure condition is met.
8. Review score, route cost, moves, and algorithm statistics.

Control principles

Every action should have immediate feedback. Disabled movement should be explained with a short message or visual cue. Do not silently deduct energy for invalid moves. Pause and restart must be available without accidental taps.

Recommended first-release rule

Treat energy as a hard movement budget. A move is permitted only if current energy is at least the entry cost of the destination. If no affordable legal move can reach the exit, the player may restart or request a hint; do not silently teleport the player.

Functional Requirements

FR-01: Start and navigation

The user can open the home screen, start a new game, choose an available level, pause, resume, and return to the menu.

FR-02: Board rendering

The app renders a rectangular grid with walls, normal floor, costly terrain, collectibles, player, and exit. Tile legends must be available.

FR-03: Player movement

The controller validates bounds, walls, affordability, and game status before applying movement.

FR-04: Resource changes

Valid movement charges entry cost. Pickups apply their effects once. The controller prevents duplicate collection.

FR-05: Pathfinding

The user can request a path from the player's current location to the exit. The algorithm must use the current board's movement costs and wall layout.

FR-06: Visualization

The app can show the search frontier, settled cells, and final route, with a fast-finish option.

FR-07: Results

The results screen shows completion status, moves, route cost, energy remaining, treasure, hint use, and relevant pathfinding statistics.

FR-08: Persistence

At minimum, settings and best scores may be saved locally. Persistence must not be required for the core game loop.

Non-Functional Requirements

- Responsiveness: normal movement should update without perceptible delay.
- Reliability: algorithm calculations must not mutate the level definition.
- Testability: pathfinding and game rules should be testable without rendering widgets.
- Maintainability: use small files and clear model/controller boundaries.
- Accessibility: use sufficient contrast, scalable text, and labels in addition to color.
- Portability: use Flutter-supported APIs and avoid platform-specific dependencies unless needed.
- Privacy: the initial game should not collect personal data or require network access.
- Recoverability: restart should reliably restore the exact starting state.

Performance note

A 64 × 64 grid is enough for a meaningful class demonstration. Optimize for correctness and readable animation before pursuing very large maps.

Game Rules and Edge Cases

Movement cost convention

Moving from cell u into adjacent cell v costs the configured entry cost of v. Therefore, opposite-direction moves can have different costs. Dijkstra supports this directed weighted graph because all costs are non-negative.

Energy cells

For the cleanest first implementation, deduct movement cost first and then apply the energy pickup, capped at maximum energy. This ordering must be used consistently in tests and documentation.

Treasure

Treasure is optional and gives points. Dijkstra optimizes cost to the exit, not treasure value. Do not claim the minimum-cost path also maximizes treasure.

Hints

A hint is advisory and does not move the player. If hints affect scoring, apply the penalty once per hint activation or define a clear alternative rule.

Failure and unreachable states

If a move is too expensive, reject it and show a clear reason. If the exit is unreachable in the graph, display a level-design error or a no-path state rather than looping forever.

Tie-breaking

Multiple minimum-cost paths may exist. Tests should assert the minimum cost and path validity, not one exact path, unless deterministic neighbor order is part of the contract.

Visual Design System: Light Theme

Design intent

Use a light background throughout. The game should look calm and polished, with strong text contrast and accent colors reserved for meaningful states.

Suggested palette

- App background: #F6F8FC (very light blue-gray).
- Surface/card: #FFFFFF.
- Primary text: #172033.
- Secondary text: #596579.
- Primary accent: #3867D6.
- Success/path accent: #138A72.
- Warning/energy accent: #D9921E.
- Danger/hazard accent: #D9534F.
- Grid border: #DCE3ED.

Terrain encoding

Do not rely on color alone. Combine color with icons, patterns, labels, or a legend. Walls should have a clear blocked texture; high-cost terrain should show its cost; the exit should have a recognizable portal symbol.

Typography and spacing

Use a consistent text scale: large title, medium section heading, standard body text, and small secondary labels. Use generous spacing, rounded cards, and restrained shadows. Avoid dense text over the grid.

Screen Specifications

Home screen

Include game title, a short premise, a prominent Play button, Continue if a saved run exists, Algorithm Lab, Settings, and credits. Avoid putting long instructions on the home screen.

Level selection

Show level cards with name, difficulty, completion state, and best score. Locking later levels is optional; the team may expose all levels for demonstration.

Dungeon screen

Top: level name, pause, score, and energy. Center: scalable square grid. Bottom: directional pad, Algorithm Vision, restart, and a compact legend. On small screens, allow the board to claim most of the vertical space.

Algorithm Lab

Show start and goal, visited/frontier cells, tentative distances, final route, total cost, and operation counts. Provide Step, Play/Pause, Speed, and Show Final Path controls.

Results screen

Show a success/failure headline, score, energy, moves, actual route cost, optimal cost from the starting state, route efficiency when meaningful, and buttons for replay/next level.

Flutter Architecture

Recommended structure

Use a lightweight layered architecture: presentation widgets, game controller, immutable or carefully controlled models, and a pure algorithm service. Provider with ChangeNotifier is sufficient; avoid introducing a large state-management framework unless the team already knows it.

Presentation layer

Owns screens, widgets, layout, input gestures, and rendering. It sends user intents to the controller and observes state; it should not calculate shortest paths inside build methods.

Game controller

Owns the current level state, player state, energy, score, movement history, completion status, hint status, and current visualization snapshot.

Domain/algorithm layer

Contains graph construction, Dijkstra, path reconstruction, route-cost calculation, and algorithm event records. This layer must not import Flutter UI classes.

Data layer

Contains predefined level maps and optional local persistence. Keep level definitions deterministic for tests and the FA demo.

Recommended Folder Structure

```
lib/\n main.dart\n app.dart\n models/\n grid_position.dart\n dungeon_tile.dart\n dungeon_level.dart\n
player_state.dart\n path_result.dart\n algorithm_event.dart\n algorithms/\n graph_builder.dart\n
dijkstra.dart\n path_reconstructor.dart\n game/\n game_controller.dart\n movement_service.dart\n
scoring_service.dart\n level_generator.dart\n screens/\n home_screen.dart\n level_select_screen.dart\n
dungeon_screen.dart\n algorithm_lab_screen.dart\n result_screen.dart\n widgets/\n dungeon_board.dart\n
dungeon_painter.dart\n energy_bar.dart\n movement_controls.dart\n algorithm_stats_panel.dart\n services/\n
progress_service.dart\ntest/\n dijkstra_test.dart\n movement_test.dart\n level_generator_test.dart\n
scoring_test.dart
```

File ownership

Assign one owner to the algorithm/model layer, one to the game controller and rules, one to UI and visual design, and one to integration/testing/documentation. A small team can combine roles, but every module should have one accountable owner.

Data Model Specifications

GridPosition

Stores row and column. Implement value equality and hashCode so positions work correctly in Map and Set collections. Provide a helper for a stable key if required, but avoid string keys when value objects are clearer.

DungeonTile

Stores tile type, movement cost, traversability, and optional collectible metadata. Prefer a typed enum for tile kind. Walls must never be emitted as traversable graph vertices.

DungeonLevel

Stores dimensions, tile matrix, start position, exit position, level identifier, display name, and optional generation seed. Validate dimensions and coordinates when a level is loaded.

PlayerState

Stores position, current/max energy, score, move count, collected-item IDs, route cost, hint usage, and game status.

PathResult

Stores whether a route exists, ordered path, total cost, settled-node count, relaxation count, and elapsed computation duration. Use a nullable or explicit status for unreachable paths rather than inventing a numeric route cost.

AlgorithmEvent

Optional event records can represent node discovered, node settled, distance updated, and route reconstructed. These events support deterministic step-by-step visualization.

Graph Construction Contract

Input

A validated dungeon level and a source position. The target may be supplied to allow early termination.

Vertex mapping

Each traversable cell is a vertex. A compact integer ID such as row * columnCount + column can be used internally, while the UI continues to use GridPosition.

Neighbor order

Use a consistent order such as up, right, down, left. Deterministic neighbor order improves reproducibility and makes screenshots and tests easier to compare.

Edge weight

For a move u to v, weight is the cost to enter v. Validate that costs are non-negative. Walls have no edges. Do not allow negative terrain costs.

Graph consistency

The algorithm and movement controller must use the same cost definition. Keep tile-cost lookup in a shared model/helper rather than duplicating values in separate files.

Validation checks

- Start and exit must be inside the grid.
- Start and exit must be traversable.
- All traversable movement costs must be non-negative.
- The level matrix must be rectangular.
- The generated path must never cross a wall or leave the grid.

Dijkstra Engine: Implementation Contract

Required data structures

- distance: best-known cost to each vertex.
- previous: predecessor used to reconstruct the best path.
- min-priority queue: selects the frontier vertex with the lowest tentative distance.
- settled set or settled flag: records finalized vertices.

Algorithm behavior

Initialize every distance to infinity and the source distance to zero. Pop the lowest-cost queue entry. Ignore stale entries whose stored cost exceeds the current best distance. For every valid neighbor, calculate candidate = distance[u] + weight(u,v). If candidate is lower, update distance and predecessor and insert a new queue entry. Stop when the target is finalized or the queue becomes empty.

Priority queue choice

Use a binary heap implementation from a well-maintained Dart package or implement a small min-heap if external dependencies are not desired. If implementing a heap, unit-test push, pop, ordering, empty behavior, and duplicate priorities.

Early stopping

Early stopping is valid when the target is removed from the min-priority queue with its current best distance. Do not stop merely when the target is first discovered.

No-path behavior

If the queue becomes empty before the target is finalized, return a no-path result. The UI should display a friendly message and provide restart or level selection.

Dijkstra Pseudocode and Complexity

```
DIJKSTRA(graph, start, goal):\n for each vertex v:\n dist[v] = infinity\n previous[v] = null\n dist[start] = 0\n pq.push((0, start))\n\n while pq is not empty:\n (cost, u) = pq.popMin()\n if cost != dist[u]: continue\n if u == goal: break\n\n for each edge (u, v, weight):\n candidate = dist[u] + weight\n if candidate < dist[v]:\n dist[v] = candidate\n previous[v] = u\n pq.push((candidate, v))\n\n if dist[goal] == infinity: return NO_PATH\n return reconstruct(previous, goal, dist[goal])
```

Complexity

With an adjacency list and binary heap, a standard bound is O((V + E) log V) time and O(V + E) space. On a four-neighbor grid, E is O(V), so the time bound simplifies to O(V log V) and space to O(V).

What to report

Report graph size, settled nodes, relaxation attempts, successful relaxations, and measured algorithm duration. Do not confuse theoretical complexity with a single runtime measurement. The runtime depends on device, build mode, grid structure, and measurement overhead.

Correctness intuition

Because all edge weights are non-negative, once the smallest tentative-distance vertex is removed with its current best distance, no undiscovered alternative route can later reduce that distance. Repeating this process finalizes shortest distances and allows predecessor reconstruction.

Path Reconstruction and Cost Verification

Reconstruction

Starting at the goal, repeatedly follow previous[goal] until reaching the source. Reverse the collected sequence to obtain source-to-goal order. If a predecessor is unexpectedly missing, treat the result as invalid rather than constructing a partial path.

Cost verification

Recompute the route cost by summing the entry cost of every path cell after the source. The source cell contributes zero for the route that begins at the player's current position. Compare this sum to the distance recorded for the goal.

Validation

- The path begins at the requested source.
- The path ends at the goal.
- Every adjacent pair differs by exactly one orthogonal step.
- No path cell is a wall.
- The recomputed cost equals the reported cost.

Important game distinction

The optimal path from the player's current position may differ from the optimal path calculated at the beginning of the level. Label these values clearly. For route-efficiency scoring, use a baseline calculated from the same starting state and under the same rules.

Movement, Energy, and Scoring Rules

Movement transaction

A movement should be processed atomically: validate game status, bounds, wall, and energy; deduct entry cost; update position; increment moves and route cost; collect any item; check exit; then notify listeners. If validation fails, no state should change.

Energy accounting

Track energy remaining and cumulative movement cost as separate fields. An energy pickup can increase remaining energy but should not erase the historical movement cost.

Scoring formula

Suggested score = completion reward + treasure rewards + remaining-energy bonus + no-hint bonus. Keep the scoring function isolated so it can be tuned without changing movement or pathfinding.

Route efficiency

Efficiency = optimal cost / actual cost × 100 when actual cost is positive and both costs use the same source, target, and movement-cost model. If actual cost is zero because the player starts at the exit, display N/A or a special completion state rather than dividing by zero.

Fair comparison

If a player collects treasure by taking a detour, route efficiency may decrease even though the score rises. This is intentional: score rewards optional goals, while efficiency measures movement-cost optimality.

Level Design and Sample Maps

Initial fixed level

Create a hand-designed, deterministic first level with a guaranteed path. Use it for automated tests, screenshots, and the FA demo. Avoid relying on random generation during the live presentation.

Difficulty progression

- Level 1: basic walls and floor; teach movement.
- Level 2: introduce rubble and compare step count with cost.
- Level 3: introduce water and energy pickups.
- Level 4: introduce expensive hazards and optional treasure.
- Level 5: larger map with several plausible routes.

Map validation

Run reachability and Dijkstra checks during level loading or generation. Reject a level if the exit is unreachable or if the intended difficulty is impossible under the energy rules.

Random generation caution

Use a fixed random seed in development. Validate every generated level before displaying it. Keep a set of hand-designed fallback levels in case a generated map fails validation.

CustomPainter and Board Rendering

Rendering responsibilities

CustomPainter should render visual state only: tile backgrounds, borders, terrain icons, exploration overlays, final path, player marker, and exit. It should not mutate game state or run Dijkstra.

Coordinate mapping

Calculate cell width and height from the available canvas size and grid dimensions. Convert a touch location to row and column by dividing by cell dimensions, after checking that the point lies inside the board.

Paint order

1. Draw base tile surfaces.
2. Draw walls and terrain patterns.
3. Draw frontier and settled overlays.
4. Draw final path.
5. Draw collectibles, exit, and player.
6. Draw optional coordinate/cost labels if enabled.

Repaint strategy

Repaint when the board state or visualization state changes. Keep the grid model outside the painter and pass immutable snapshots or stable references. Avoid creating expensive objects for every cell on every frame if the board grows.

Mobile layout

Use LayoutBuilder or constraints to compute a square board area. Keep controls outside the canvas so they remain accessible and are not hidden by gesture handling.

Algorithm Vision: Animation Design

Two modes

Fast mode computes the result and displays the final path. Step mode replays recorded algorithm events so the evaluator can see frontier growth, node settlement, and distance updates.

Event model

Represent events as typed records, for example NodeDiscovered(position, distance), NodeSettled(position, distance), DistanceUpdated(position, oldDistance, newDistance), and PathCompleted(path, totalCost).

Animation architecture

Run the algorithm to produce a deterministic event sequence, then let the UI advance an event index. Do not add arbitrary delays inside Dijkstra itself. This keeps the engine fast, deterministic, and independently testable.

Controls

- Step forward.
- Play/pause.
- Adjust speed.
- Show final path.
- Clear visualization.

Avoiding misleading visuals

A discovered node is not necessarily finalized. Use distinct styles for frontier and settled nodes. The final path must be drawn only after the target is finalized and the predecessor chain has been reconstructed.

Game Controller and State Transitions

Controller responsibilities

The controller receives user actions and updates the game state. It is the only component allowed to apply movement costs, collect items, change score, complete levels, or restart a run.

Suggested states

- ready: level loaded, no movement yet.
- playing: player can move.
- paused: movement disabled.
- showingAlgorithm: visualization controls active; movement policy is explicit.
- won: exit reached.
- lost: failure condition reached.

State transition rules

A won or lost game cannot accept movement. Restart creates a fresh run from the immutable level definition. Pausing should preserve position, energy, score, and history. Algorithm visualization must not mutate the player's position.

Avoiding stale state

When requesting a path, snapshot the current position and level costs. If the board can change during computation, invalidate or recompute the result. In the initial static-level design, only player position changes, so a new hint can simply use the latest position.

Local Persistence and Settings

Minimum persistence

Persistence is optional for the core game, but useful for a polished release. Store sound preference, control preference, unlocked levels, and best scores. Do not persist transient algorithm animation frames.

Data format

Use simple JSON-compatible values with stable level IDs. Store schema/version information if the data structure may change during development.

Failure handling

If saved data is missing, corrupt, or incompatible, initialize safe defaults rather than preventing the app from starting.

Privacy

The first release needs no sign-in and no network connection. Avoid collecting personal information. Clearly state that progress is stored locally if this is included in the UI.

Testing Strategy and Test Matrix

Algorithm unit tests

- Single straight corridor with known cost.
- Two routes where fewer steps cost more energy.
- A destination surrounded by walls.
- Start equals goal.
- Several equal-cost shortest paths.
- A maze with expensive but traversable terrain.
- Invalid negative movement cost rejected during validation.

Game controller tests

- Valid move deducts correct cost once.
- Wall move changes no state.
- Out-of-bounds move changes no state.
- Insufficient-energy move is rejected.
- Treasure is collected only once.
- Energy pickup respects maximum energy.
- Exit transitions to won state.
- Restart restores the initial state.

Integration tests

Load a known level, ask for Algorithm Vision, and verify that the displayed route is valid and its cost matches the result. Then manually follow the route and confirm that the cumulative movement cost agrees with the algorithm's cost, subject to the source-entry convention.

Performance Measurement and Metrics

Metrics

- V: number of traversable cells.
- E: number of valid adjacency edges.
- Settled nodes.
- Relaxation attempts and successful relaxations.
- Path cost and path length in cells.
- Algorithm elapsed time.

Measurement method

Measure only the algorithm call, not animation, rendering, or screen transitions. Run multiple repetitions for each grid size and report a median or average with the device/build mode identified. Very small timings can be dominated by timer resolution.

Comparison

A useful extension is to compare Dijkstra with breadth-first search on uniform-cost maps. BFS is appropriate for minimum number of edges when all edges have equal cost; it is not a correct substitute for Dijkstra on arbitrary weighted maps.

Honest reporting

Do not promise a specific runtime or claim a benchmark proves theoretical complexity. Explain that measured time is an empirical observation, while complexity is derived from the algorithm and data structures.

Error Handling and Debugging Guide

Common defects

- Path crosses a wall: graph builder or neighbor validation bug.
- Path cost differs from energy usage: inconsistent destination-entry cost rule.
- Algorithm stops too early: stopping when target is discovered instead of finalized.
- Animation shows wrong distances: events are recorded before the update or replayed out of order.
- Player moves twice on one tap: duplicated gesture/button callbacks.
- Restart leaves old overlays: visualization state was not reset.
- Level generation hangs: no bounded retry limit or no guaranteed corridor.

Debug instrumentation

In debug mode, log source, target, settled count, relaxation count, final cost, path length, and no-path status. Avoid logging on every animation frame in release builds.

Recovery policy

A failed pathfinding result should not crash the game. Show a friendly message, preserve the player state, and offer restart or level selection.

Security, Accessibility, and Quality

Security and privacy

This offline game has a small security surface. Avoid unnecessary permissions, network access, analytics SDKs, or account collection. Keep dependencies limited and review package maintenance before adding them.

Accessibility

- Do not encode terrain solely through color.
- Support readable text and high contrast.
- Provide labels for directional controls.
- Use sufficiently large touch targets.
- Offer reduced animation or instant visualization.
- Ensure the game can be understood without sound.

Quality principles

Use linting, format Dart code consistently, keep algorithm methods pure where possible, and write tests before visual polish. Do not allow a UI shortcut to bypass movement validation.

Team Roles and Collaboration Workflow

Suggested role allocation

- Algorithm owner: graph builder, priority queue, Dijkstra, correctness tests.
- Game logic owner: movement, energy, collectibles, score, controller.
- UI/visual owner: light theme, board painter, menus, controls, results screen.
- Integration and QA owner: level maps, integration tests, device testing, report, demo.

Team workflow

1. Agree on model classes and method signatures before parallel coding.
2. Create a small level fixture and share it with all contributors.
3. Merge algorithm and game logic before adding complex animation.
4. Run tests after every integration change.
5. Freeze new features before the final demo and fix only defects.

Definition of done

A task is done only when its code is integrated, formatted, tested, and demonstrated. A screenshot or isolated file is not sufficient if the feature does not work in the app.

Milestones and Suggested Schedule

Milestone A: specification and algorithm

Finalize movement cost convention, model classes, level format, and Dijkstra contract. Deliver passing unit tests for known maps.

Milestone B: playable prototype

Render one static level, add movement controls, energy, walls, exit, and restart. Confirm all state transitions.

Milestone C: algorithm visualization

Add final path overlay first, then event-based step visualization and statistics.

Milestone D: polish and progression

Add level selection, scoring, treasure, instructions, and responsive layouts.

Milestone E: quality and submission

Run tests on the target device, capture screenshots, prepare the explanation, and verify that the demo works offline.

Schedule guidance

Plan in deliverable-based milestones rather than estimating only by coding hours. If time is short, postpone procedural generation, audio, persistence, and advanced scoring before sacrificing algorithm correctness or a complete game loop.

Build, Run, and Integration Checklist

Environment

- Install a stable Flutter SDK and verify with flutter doctor.
- Create the project and commit the initial scaffold.
- Confirm the app launches on an Android emulator or physical device.

Dependency discipline

Add Provider and a priority-queue package only if the team chooses those implementations. Check current package versions using the team's installed Flutter environment rather than copying stale version numbers from a report.

Integration order

1. Compile models.
2. Run Dijkstra unit tests.
3. Render a static grid.
4. Connect movement controller.
5. Connect pathfinding result.
6. Add animation and results.
7. Build a release APK and test it on the presentation device.

Final checks

- No network access is needed for the demo.
- Device orientation and screen size have been tested.
- The game can be restarted quickly.
- A known fallback level is available.
- The build installed on the device matches the source revision used for testing.

FA Demonstration Plan

Recommended 4–6 minute flow

1. Introduce the problem: minimum number of steps is not always minimum cost.
2. Show a level with normal and costly terrain.
3. Move manually and show energy deductions.
4. Activate Algorithm Vision and explain frontier, settled nodes, and relaxation.
5. Reveal the final path and verify its total cost.
6. Show the results panel and complexity dashboard.
7. Explain time and space complexity and show one unit test.

Live-demo safeguards

• Use a deterministic, tested level.

• Keep a prebuilt APK available.

• Avoid random generation during the core demonstration.

• Prepare screenshots or a short recording in case the device fails.

• Do not claim that one measured runtime proves asymptotic complexity.

Viva questions to prepare

Why Dijkstra instead of BFS? Why must edge weights be non-negative? What is relaxation? Why use a priority queue? What is the complexity? How do you detect no path? How do you verify the returned path cost? What changes if obstacles move?

Risks, Trade-offs, and Future Scope

Risk: over-scoping

A game can expand quickly into animation, enemies, procedural generation, sound, and persistence. Protect the core algorithm and complete loop first.

Risk: algorithm hidden behind visuals

A polished game is not sufficient for DAA assessment if the algorithm cannot be explained. Keep a clear Algorithm Lab and provide a small trace of distance updates.

Risk: inconsistent game and algorithm rules

Use one source of truth for tile costs and movement validation. Add tests that compare algorithm cost with route cost recomputation.

Trade-off: pure Flutter versus game engine

Flutter widgets and CustomPainter are sufficient for a grid puzzle with turn-based movement. A dedicated game engine can be considered later if continuous animation, physics, or many moving entities become essential.

Future extensions

- Compare Dijkstra and A* on identical maps.
- Add BFS for uniform-cost maps.
- Introduce fog of war.
- Add backtracking-generated mazes.
- Add bounded enemy movement with replanning.
- Add optional challenge levels and local achievements.

Conclusion and Handover Checklist

Algorithmic Dungeon: Escape the Labyrinth is feasible as a Flutter FA project when its scope is controlled. Its strongest academic feature is the direct relationship between a weighted-grid game and Dijkstra's algorithm. The project should prioritize correctness, consistent cost accounting, a responsive board, and an explanation that the evaluator can follow.

Team handover checklist

- All team members understand the game rules and movement-cost convention.
- Model and algorithm interfaces are agreed.
- Dijkstra tests pass before UI integration.
- The demo level is deterministic and reachable.
- The visualized route matches the reported cost.
- Energy, scoring, and route efficiency are documented separately.
- The app runs offline on the target device.
- The report, screenshots, source code, and demo build are ready.

Final recommendation: build a reliable core game first, then add visual polish and advanced features only after the algorithm and game-state tests pass.

Appendix A: Suggested Dart Contracts

Algorithm service contract

```
abstract class PathfindingService {\n PathResult findPath({\n required DungeonLevel level,\n required
GridPosition start,\n required GridPosition goal,\n bool recordEvents = false,\n });\n}
```

Controller action contract

```
abstract class GameActions {\n void move(Direction direction);\n void requestHint();\n void pause();\n void
resume();\n void restartLevel();\n void loadLevel(String levelId);\n}
```

Design notes

These are interface sketches, not a complete drop-in implementation. Final signatures may change to match the team's models. Keep return types explicit and represent no-path and invalid-level states clearly.

Appendix B: Example Acceptance Test Cases

Case 1: cheaper longer path

Given a map with two routes, one having fewer steps but high-cost tiles and another having more steps but low-cost tiles, Dijkstra must return the route with lower total cost.

Case 2: wall collision

Given a player adjacent to a wall, an attempted move into the wall must leave position, energy, score, and move count unchanged.

Case 3: no path

Given a wall arrangement that disconnects the exit, the pathfinding service must return no-path and the UI must remain usable.

Case 4: energy pickup

Given a valid move onto an energy cell, the movement cost is deducted first, then the pickup is applied once and energy is capped at maximum.

Case 5: route cost

For every returned path, independently sum the entry costs after the source. The sum must equal PathResult.totalCost.

Case 6: restart

After movement, treasure collection, and hint use, restart must restore the initial position, energy, score, collectible state, route history, and visualization state.

Appendix C: Submission Package

Recommended deliverables

- Flutter source project.
- README with setup and run commands.
- This project report.
- Screenshots of home, gameplay, Algorithm Vision, and results.
- Unit test results.
- Short demo video or backup screen recording.
- Presentation slides explaining graph modeling, Dijkstra, complexity, and test evidence.

README should contain

1. Project overview.
2. Requirements and installation.
3. How to run the app.
4. Game controls and rules.
5. Algorithm explanation and complexity.
6. Test command and known limitations.

Final handover

Before submission, remove debug-only controls that could confuse the evaluator, but retain a way to access the algorithm visualization. Verify the project from a clean checkout so the app does not depend on files or packages that exist only on one team member's machine.